BACKEND INTERVIEW PREPARATION

SpendWise
10 Essential Questions

Short, natural answers based on the original backend-ledger project. Designed to be spoken clearly in an interview without memorizing complicated terms.

30-second project introduction

“SpendWise is a personal finance backend built with Node.js, Express and MongoDB. Users can register, create accounts and transfer money. Instead of storing a changing balance, the system calculates balance from credit and debit ledger entries. I also used JWT authentication, database transactions and idempotency to make the APIs safer.”

Easy answer rule: Say what it does, why you used it, and one small example. Stop after that unless the interviewer asks for more.

The 10 Most Important Questions

1 What problem does this project solve?

It provides backend APIs for a personal finance application. A user can register, create accounts, check balances and transfer money. The main goal was to manage financial records safely and keep a clear history of every balance change.

Remember: User → Account → Balance → Transfer.

2 Which technologies did you use?

I used Node.js and Express to build the REST APIs. I used MongoDB as the database and Mongoose for schemas and database operations. JWT handles authentication, and bcrypt hashes user passwords.

Remember: Express API, MongoDB data, JWT auth, bcrypt passwords.

3 How is the project structured?

I separated the code into routes, controllers, models, middleware and configuration. Routes define API URLs, controllers contain the business logic, models describe the data, and middleware checks authentication. This separation keeps the code easier to read and maintain.

Remember: Route receives → middleware checks → controller works → model stores.

4 How does authentication work?

During registration, the password is hashed with bcrypt before it is stored. During login, the entered password is compared with the hash. If it matches, the server creates a JWT containing the user ID. Protected routes verify this token before allowing access.

Remember: Hash password → login → issue JWT → verify JWT.

5 Why did you use a ledger instead of storing the balance directly?

A stored balance can be changed accidentally and does not explain how it was created. My ledger keeps every credit and debit entry. The balance is calculated as total credits minus total debits, so every change has a history that can be checked.

Remember: Balance = Credits − Debits.

6 Why are ledger entries immutable?

Immutable means an entry cannot be edited or deleted after creation. This protects the financial history. If a mistake must be corrected, the safer approach is to create an opposite entry instead of changing the old record.

Remember: Never erase history; add a reversal.

7 How does a money transfer work?

The API first validates both accounts and checks the sender's balance. It creates one transaction, adds a debit entry to the sender and a credit entry to the receiver, then marks the transaction completed. Both ledger entries belong to the same transfer.

Remember: Validate → debit sender → credit receiver → complete.

8 Why did you use a MongoDB transaction?

A transfer changes more than one document. A database transaction makes these changes all-or-nothing. If one step fails, MongoDB rolls everything back, which prevents money from disappearing or appearing in only one account.

Remember: Both changes succeed, or neither change happens.

9 What is an idempotency key and why is it needed?

It is a unique value sent with a transfer request. If the same request is sent again because of a retry or double-click, the server finds the existing transaction instead of creating another one. This helps prevent duplicate payments.

Remember: Same key = same transfer, not double payment.

10 What was the main challenge and what did you learn?

The main challenge was keeping balances correct during transfers. I solved it with immutable credit and debit entries, MongoDB transactions and idempotency keys. I learned that financial systems need consistency and traceability, not only basic CRUD APIs.

Remember: Correct balance, safe transfer, clear history.

If you forget an answer

Pause and return to the basic flow: “The request is validated, the user is authenticated, the database operation is performed, and a clear response is returned.” Speak slowly and explain only what you understand.